

ActInf ModelStream #004.1 ~ "Implementing Active Inference by Message I

ModelStream | 1:40:11

a signal processing group

and so i designed

my training

sorry actually i i just one thing

changed could you just restart that but

go for it

okay

uh thanks for the introduction then my

name is bertie fries i'm a professor in

eindhoven at eindhoven

university of technology in the

netherlands

i'll be a part of

an electrical engineering department in

a signal processing group so we designed

signal processing algorithms and about

six seven years ago i read

for the first time a paper by carl

fiston it was called a rough guide to

the brain

and struck me that this could be

fantastic for signal processing so since

then i'm really trying to work with

people in my lab

on realizing realizing agents that will

design will automate the design process

of signal processing algorithms and as

you know we're doing this by message

passing and we want to talk about that

today

uh thais

yeah um my name is tyson i'm a poster

with bird's lab i did my phd also an

active inference on how to automate

those processes and uh also together

with michael cox and all the colleagues
from the lab we built the toolbox called
fornilab and i'll be talking and walking
you through um how we apply that in an
active infra con inference context um
and do some cool things with that
and so that's for later

yeah hello everyone so my name is nitri
bhagav i'm a phd candidate in bias lab
also in hue in university and yeah my
work is mostly about reactive message
pricing based bayesian inference
that we hope will help with active
inference as well but that's also for
later i will talk about it

on my

uh time yeah my slot

cool thank you

okay um

shall i uh go then and do a little

introduction a few slides

let's see that

yeah okay um

let's see if i can share my

slide

yep looks good

yeah okay

so

the

first slide is uh eindhoven

um because you may wonder where is i

know in relation to amsterdam well it's

about

100 kilometers south of amsterdam close

to the belgian border and not so far

away from germany either

um

so it's a it's sort of a high-tech city

phillips

originated and opened on the right

bottom you see a picture of the center

and the right top is a view of the

campus of our

university of technology

here's an aerial view of groups

going through five here's an aerial view

of our campus

and uh so

here let's see if i can share a pointer

yeah so uh this is the building for

electrical engineering so this is where

we are

ios lab is short for bayesian

intelligent autonomous systems that's

what we try to build

we have about three

let's say staff members faculty members

and currently six phd students dmitry is

one of those pc students

and we have open positions if there are

people watching that are interested in

probabilistic programming or how to make

active inference work

then what are we trying to do this is a

picture that's probably familiar to

everybody in this uh forum right this is

uh this wants to show that well the only

thing that's really going on in the

brain is

for energy minimization or expected free

energy minimization

to do everything

and that's a huge inspiration to us to

us engineers so what we try to do is

basically this we want to put this in an

iphone or on a raspberry pi

and let a robot
learn how to ride a bike
but the beauty of this framework for
engineering purposes is that it's almost
one solution approach to to to any
problem so
if we can do it you could teach a robot
how to write a
energy minimization probably we can also
apply this in virtual reality and design
algorithms for hearing aids or even
self-driving cars
the the big
let's say
promise or the attractiveness for
engineering is that it it's just one
always the same thing you just have to
propose a model and minimize the free
energy
no matter what the application is but
it's very uh
appealing
the problem for engineering is that this
energy functional
is a function of observations and
observations are streaming data coming
usually
well could be at
every millisecond so it's a highly time
varying function
and the number of latent variables let's
say the space of latent variables is
usually very high so we have a very high
dimensional function that is time
varying
and
we want to minimize that now the brain
is very good at it right the brain has

uh what is a 10 to the power of 14
synapses 100 billion euros
but a normal
optimization library and model up or
python will not cut it you cannot
minimize a time varying function of
thousand variables in
matlab it's not going to work
so we need something quite radical here
and the idea we go with is not that
radical again we just take inspiration
from the brain right the brain is a
network of message passing and turns out
in my own field
in signal processing
or information theory to be exact
this has been formalized there's a paper
here by
dave forney in 2001
codes on graphs normal realizations he
called it normal factor graphs but in
honor of david forney there today
they're called forney style factory
graphs
and
so this is the origin of the factor
graphs that we are talking about and a
few years later
hans andreia luliger at the university
in series
uh i have made this
popular in the signal processing
community and already around 2007 you
see in in his papers these uh
yeah these typical structures like this
that we will show also later these are
kalman filters
this is really also what tristan talks

about these kinds of structures so
um this is uh
what we want to do in our language is to
combine what
active inference from the inspire
inspiration from the neuroscience
community and combine it with what we
know in signal processing and
information theory about factor graphs
and use these tools to to implement this
uh so today
we have two presentations one by text
from the line
uh on how to do message passing by
forney lab forney lab is the toolbox
that we wrote
uh you know
the name of course refers to the forney
uh and thais will show how to do active
influence with this toolbox
then we're also working on a new version
it's called reactive message passing and
dmitry is the main person there and
dimitri also
i'll talk about that a bit
and that's all for me
um i'll give it back to you dan or maybe
uh
can continue what is your preference
i'll
stop sharing my screen
see
super interesting thanks a lot for the
context and
it will be awesome to hear from the
authors with maybe any reflections on
those general points you raised and how
it gets applied in a really specific way

because you brought up a ton of topics
that we talk about all the time like the
simplicity of one unified approach and
the challenges of ongoing optimization
problems and drawing inspiration from
nature so great things i think we can
just jump to whichever
of the presentation

you all prefer

all right then i'll go ahead and try to
share my screen

see if that works

[Music]

select windows screen entire screen okay

a lot more

practical streams i think you can see my
screen now yep

you can make it look just large on your
side and i'll resize it there we go

looks great

there we go all right

well thanks again for having us can you

click away i will be giving a little

introduction sorry where it says you're

sharing your um screen you could hide

that little jitsi thing

um and it's gone

thank you

so um yeah i'll be giving a little

introduction on how to

do message passing with phony lab in the

context of active inference

and to give a little bit of motivation

for this talk um this is kind of the

situation where we're interested in so

we have some kind of an environment

and we have an environmental process

that's running in that environment and

we want to develop an agent that has
some purpose or does some purposeful
task within that environment
and the agent is allowed to
send actions to the environment and
manipulate that environment and it will
receive observations
and in the environment process there is
some
function r_t that is running there and it
might be simulation that we run that we
interact with that or it might be even a
real world process of which of course we
don't really know what r_t is running in
the real world and somehow we want to
build an agent that does something
in that world
so where do you start with that
there's this paper from conant and ashby
in the 1970s
and they have an interesting theorem
that says that
every good regulator of a system must be
a model of that system
so
that actually means that if you want to
build an agent
that regulates or
manipulates our system the environment
then we have to model that system
so as an
engineer we are in the business
of building models
generative models to be precise
and this model represents our belief
about
how observations
follow from our manipulations

we represented by a function f that is a
function of y the observations
the controls u
and also some latent state hidden
variables x

that act as intermediate between
observations and controls
now

reasoning forward is one thing but in
the end what we want to do is observe
things and then propose controls that
lead us to favorable states

and that is where

the idea of the free energy principle
and active inference comes in

so if we want

to see this is the hypothesis so that if
we want to build an agent that does
something purposeful then we can do that
by minimizing a free energy bound on
surprise

so we want to build an agent that avoids
surprises and by that we can do bayesian
inferences about that environment

so we have to build some kind of free
energy functional

for the agent to optimize

and by that duration inferences for
actions and controls

[Music]

and we defined as for energy functional
as a kl divergence between the
generative model that we have
and some kind of an approximate
posterior that we postulate that we here
define q_t

and minimizing this energy functional
will then allow us to reason backwards

from observations towards controls
that's kind of the general idea
and
having postulated this i have choices to
make
so for example one big choice that i
want to make is how do i choose my model
how do i choose my f
in such a way that
is useful to me
and also how do i choose my qut
how do i choose the factorization of
that
such that i can make my inferences
and
that process is kind of a thing of trial
and error
and um
as an engineer i want of course to build
the best model that i possibly can
but how how do i go as quickly as
possible through a process of trial and
error that gives me the best mark the
best model
and this is where the idea of a model
design cycle comes in
and this was again made popular by david
bly
in his paper in 2014 built compute
critique repeat
and he proposes a cycle where an
engineer proposes a model
and then with that model you want to
infer your quantities of interest given
data
and if you have infer the model like
that you want to criticize it
and based upon

well the performance that you have
evaluated you want to reiterate rebuild
your model
infer
again
and see how well it then performs
until you're satisfied and you can apply
it in your agent in in practice maybe in
a real world setting
and then the challenge becomes then how
do i go through this design cycle as
quickly as possible
so we want to be flexible we want to
automate things
and making model proposals itself is
something
that well you have to do as an as an
engineer you have to come up with a
proposal of
how you believe the world functions
but once you have that
everything else can be automated
you can infer these quantities by
probabilistic programming you can
evaluate the free energy as a measure of
model performance
[Music]
and
automating this cycle will be the key to
making model proposals for an agent that
will be useful in practice so you want
to go through this as quickly as
possible
so that's why we choose a specific
method we choose vector graphs
to represent our factorized models
because
in a graph you can do manipulations you

can add nodes delete nodes you can
rewire things very
quickly and once you have a graphical
representation of your model you can do
message passing on that graph in order
to automate the inference
and you can even evaluate free energy by
local contributions in that
graph so that is why we choose a
graphical representation and that is why
we want to do message passing because we
want to design models quickly
so that we can design effective agents
now how do these graphs
kind of work right how do you build an
effective graph or what does that look
like
so we choose a specific representation
of our model
and this is an example where we have a
generative model of five variables x_1
through x_5
and we have three factors f_a , f_b and f_c
and here on the picture on the right is
the graphical representation of that
model
and as you can see the edges correspond
to the variables and the nodes
correspond to the vectors
and the edges are connected to the nodes
of which
the vectors are in arguments so here you
see node f_a which connects edges x_1 and
 x_2 well that is because x_1 and x_2 are
arguments of f_a
same for f_b and f_c
now
how do we do inference in this graph

so suppose that we observe x_5
which is indicated by this little solid
square
and we want to compute our belief over
 x_2 given x_5
well what do you do well you marginalize
everything and
by marginalization i mean you integrate
out every variable
except the one that you're interested in
so accept x_2
you integrate everything out and you you
add this constraint that you observe x_5
now if you have a big model
then this integration becomes very
cumbersome because this here is
you have here four variables of which to
integrate
so
this room this this integration space
that really explodes if you want to do
this naively
but you can be smart about this
you can
you can
re-shuffle these integrals according to
their respective terms
so for example
this integration over x_5
well there is only one factor where x_5
is an argument of
so i can use the distributive rule to
bring this integral inside
so that i divide these integrals up into
smaller parts which are manageable
and this is where message passing comes
from
message passing is essentially solving

these integrations one by one
so i can first integrate out x
 x one here for example
which
as we say summarizes all the information
inside this orange box and it gives us a
message first message here that exits
this orange box
then it can continue for the bottom so
integrate out x_5
and i get a second message
and i can use this second message
in the computation of the third message
so this way you get
a nested solution approach
where i use the the the solution that i
have for the message two in the
computation of message three
and you see that in the end
the multiplication of these two
colliding messages on the edge for x_2
will give me
my proportional belief my proportional
posterior
over x_2 given what i have observed
so to give a little example of
a specific node how that works
here's an equality constraint node
which we use
as
as a constraint
to
to constrain the beliefs upon the the
adjacent edges to be equal
um and what that does is that they will
follow if we if we go through the math
here
um

this is what the node function looks like so i say okay it's a function of three variables x y and z and i constrain z to be x and i constrain z also to be y so then x will also be y if i constrain those two to be the same so let's see if we can derive messages based on this so if i want to summarize the information into this orange box i can use the sub-product rule for that i have the two messages coming in here from the left and from the bottom message one and message 2. i multiplied that by the node function of the equality node and i want to perform this integration now i substitute this this node function here with dividing integration and then i can use the shifting property to kind of replace the arguments here of x and y with the arguments of z so what i have essentially done here is saying i can compute this message three by multiplying the two messages that are incoming from the left and from the bottom now if you squint your eyes a bit and you look uh you look through your eyes you can kind of recognize a proportional base rule in here

so essentially you can say well if
message three here represents my
posterior
then message one can represent my prior
a message to my likelihood
and then i have a proportional posterior
message three and then i can pass on to
the rest of my model

and this node is also often used to kind
of combine information that's coming
from the left and from the bottom
um

so this is a very quick introduction to
do message passing

um we have derived lots of rules also
from the literature also implemented a
lot of rules that are already derived
into uh into phony lab which is
basically

well a software problem programming
suite that

does the scheduling and
this message computation for you
so you don't have to think about that
too much how to redistribute these
integrals and how to derive

well a lot of updates for specific nodes
um so that's kind of

the things that that we can we can
automate

uh on that side of the uh of the design
cycle

now

other any questions about this because
i'm going through this very quickly and
what i'm going to do next is kind of
want to walk through an example
so maybe this is kind of a natural point

where people have questions that they
can kind of stop me and think hey you're
going to fast here
i think it'll be good to see the example
and then we have some questions in the
live chat and also anyone's welcome to
add more
all right sure sure
so how do you do this in practice
in an active inference context so this
is a little example it's called the
bayesian thermostat
where we have an environment environment
so
where we have a heat source here on the
left and we have a little car that can
move around it can move away from the
heat source or it can move towards the
heat source
now the position relative to heat source
is what we call x
and at every position there is a
specific temperature
um
and temperature that we measure at a
specific position is what we call y
and we have a preference of being at a
temperature of four
 y four i don't know it's something we
choose
now we control control this car by
moving it left or right and our control
this is the velocity that we move left
or right with and it's called
 u
so this is our environment
and we want to build a model for that
so

in active interference what we want to
do is
we want to reason
from states where we would like to be
towards controls that we have to apply
at this current at this current time in
order to get there in the future
so what we want to do is we want to
reason forward from the current time
time t
up into
some time horizon big t
and
in the future we want to consider
how the agent will move
and where we want that agent to be at
that time and in the end we will want to
be at a temperature of four
and that's a state space model that
represents our belief for how this agent
will
will
move itself through the world
so at time t we have an observation
coming in of a specific temperature
and maybe we have already made an action
 u_t and
we say that
the current state x_t relates to the
previous state and action that we have
by adding so basically u_t here is a
velocity that we add to our current
position
 u_t in in one time step and then we get
kind of our next position
by just adding that and also applying
some gaussian noise
and so we have a state transition that

is this additive with uh
with duty saying i want to move forward
to the left or four to the right um and
then we add some gaussian noise saying
okay we're kind of uncertain about
um how this agent might move in the
environment
we have an idea about that but let's add
some variants in order to
account for some uncertainties
and then we say here with this vertical
line that if we have
um an observation
or if we have a position then it relates
to an observation of the temperature
well
-1 it's kind of a rough guess it's just
saying well
if i move to the right if i move away
from my heat source then my temperature
will decline
with a slope of minus one
and it's kind of a really rough guess in
the real world there will be a very nice
temperature gradient that maybe will
move like
will move very slowly downwards
um but we're just saying hey we don't
really know what that is let's make a
rough guess we have a belief this is our
generative model our belief about the
world let's just say that temperature
decreases with position
and again we're not very certain about
that so let's again add
some gaussian noise
and now i extend this
into the future so i say actually well i

believe that in the future my
environment will also evolve
according to this
but
i also have some ideas about where i
want to be
and this is where gold priors come in
that constrain my future observations of
the temperature to be around this
desired temperature of four
and i'm saying then here with some added
noise well um
i want to be around four right it
doesn't have to be exactly but i want to
be pretty certain that um i will be
around four at time t plus 1 and i will
continue
to remain there
until
sometime in the future
now we have regenerative model
and
we can define our energy functional and
we can solve this for energy functional
by message passing on this graph
and we pass these messages towards
our next control
because we're interested in inferring
what we have to do next like from from
from the next time
um and this is where we can do message
passing we can summarize all the
information that we have from
from the past so this will be an
estimate of the current state of the
agent
and it will be biased by our beliefs
about the future saying where we want to

be
and that will influence or determine
which control that we will take
at the present time
or at the next time
all the details of that this are
are described in
this paper from 2019
by
you can also see kind of how we apply
that
within the action perception loop
and things like that
because you will have to do this at
every time t
because your state estimate will change
the word will change according to your
actions so you will for every time you
will have to recompute a new action for
what you're going to do next
um and this is kind of the
the main idea now of course we don't
want to do everything
of this by hand so what we want to do is
well we want to use a tool for that
so now
i'm going to show a little demo of how
you would implement this
um using the the formula probabilistic
programming tool
so let's see if i can
open our little demo so this is uh
this is readable actually or should i
zoom in more
um
you can zoom in a little bit more
maybe like this
uh yeah i can resize it

um

that that's pretty good

oh that's perfect

that's pretty good yep thank you

so this is um so so it's kind of set

what what we want to do

but so this is kind of our definition of

the environment itself so this is the

the real world

well we have this this temperature

gradient which is a very nice function

so very smooth function

where at the heat source we have a

temperature of 100

and it

it goes down with

with with position from the heat source

[Music]

so this is a real world we don't really

know this so this is the one that we're

going to approximate with this

very rough minus one and then

we'll see whether that will work on it

now this is my wonderful ascii art that

has now been resized of basically the

model that i've just shown in the slides

um

and this here is our model definition so

actually i have to zoom out a bit to

make it more readable

yeah so we want to build a graph for

this right so um here we have our

observation states controls

and we define a prior belief

about t minus one so where have we been

which we're going to say is a gaussian

and with some mean and some variance

and then for every time into the future

from uh from one until the horizon
we will have a prior belief about
controls which basically says well
what am i allowed to do when i say okay
well

um

i have a i'll have a gaussian prior about
controls with

a mean of zero

i'll substitute that there and then

maybe some variance

and um

here i say well i have my position of
the agent it relates to my controls by

just

this addition

um

with some added precision gamma so this

is our transition model

this is horizontal

line

and this is our observation model which

is very

rough estimate minus one of the

temperature gradient

times current state and also some

some some precision

conservation precision

and this represents my goal prior

so note that i haven't really put in any

specific values here yet i've just have

placeholders for actual statistics and

values that i'm going to put in

because the actual derivation of my

algorithm doesn't really depend upon the

statistics it's just something that

we can we can put in later

so this is what i want to do build a

free energy functional and the phony lab

is actually just

basically four lines of code

um so this stereo factorization that

defines my factorization for q

saying that um

i'm just going to have

a joint variational distribution

um over the entire graph

i don't really subdivide it into a

structure into a structured

factorization some people are familiar

with that

um but just saying okay i just have this

stereo factorization of the variational

distribution to be my entire graph

and never just one command

i can derive a message passing algorithm

that propagates all the messages towards

control u_2 which which which is the

control for the next

time step t plus one

and i want to evaluate my free energy so

also give me at the same time an

algorithm that i can i can execute in

order to evaluate my free energy

and this line will convert this

to

two source code that i can then load in

my environment

and what this does is it builds a

message passing algorithm for us

so this is this this is the code that

executes the message passing on our

graph

and here you can see that it computes

all these messages

by some product rules and other rules

there we have pre-derived and we have implemented in fornilab and you can see that these messages they depend upon the statistics so this is for example the mean of our prior over states and variance of a prior overstates and we have other messages that depend upon previous messages in the way that i explained message passing and this builds an entire list of in this case 26 messages and in the end we're interested in the marginal beliefs so the posterior marginal beliefs about about for example or controls and you get these beliefs by multiplying messages together so for example to get a new belief over the following control you multiply messages 9 and 25 and you also recognize this from a little introduction past introduction where you have colliding messages on the edge that you multiply in order to get your posterior beliefs and that's what we return in the end that's what we're interested in so this is the execution of that algorithm and notice that we've made an entire schedule here and they can be very cumbersome can be very long very long schedule and so that makes it expensive maybe to load and to execute

and dimitri has the solution for that
and he'll explain that in a minute
um so i'm very happy about building
schedules
and um dimitri not so much uh
and he tries to find uh find a solution
for them get rid of them
um because in the end that will be the
fastest
to really don't
have a schedule that not really had
recipe but just cook by whatever is
given to you
so now we execute this algorithm in
practice so this is our action
perception loop for every time step
we want to
we want to act we want to evoke an
action from the agent we want to execute
that that that action in
the uh in the environment so this is
sending the action towards actual real
world or kind of simulated world in this
case and what we get from that is an
observation
and from our action observation we want
to infer
infer our new action
and also return
the variation of free energy
um
and then prepare for the next step so in
every step we propose an action to the
environment it actually gets executed we
observe a new outcome
we we infer new action
um and we evaluate how well we did with
that action

and then you get well nice plots like

this so

this is the velocity of our agent

we start at zero and you can see that it

moves very quickly to the right

um

and then

it also moves back a little bit um until

the end comes to a stationary point more

or

less um and you can see actually here

what it does so it starts out at a

temperature of 20

then it moves very quickly to the right

away from the heat source

it overshoots its mark

and then it actually goes back and then

it settles around this desired

temperature of four that we have uh

encoded by by our goal priors

um so why does overshoot well our model

of the environment wasn't perfect right

we had this very rough estimate of minus

minus one

um and in the real world was this very

nice smooth kind of complicated function

um

that has this kind of bell shape

so even though

our generative model of the environment

wasn't perfect

um in the end we paid by that pay for

that by overshooting a little bit but

still we were able to get to our desired

state at the temperature of four so in

the end

it kind of worked pretty well

and you can also see that the free

energy of the agent also decreases with time so we start off at around a thousand so the scale's logarithmic here and it decreases very quickly um to our sum value it's rather rather low right and then we have some noise that that induces some unexpected changes or some unexpected surprises um so that's why you have these little ripples these little ripples here but in the end we're minimizing free energy and it's going it's decreasing drastically so we actually have this free energy minimizing agent um and yeah that's uh that's basically all i have the story and and the demo that was awesome thank you very much we can uh with screen share on or not ask a few of the questions from the live chat and also give people a few seconds to type more so i'm just going to jump into the questions first let me stop my screenshot great so we're still sharing the screen story um we see just the jitsies so go just click the screen share button again in jitsi so awesome thank you so the first question is from john and john writes how does the factor graph approach work if the graph structure is not known beforehand is structure learning possible uh yeah so so that's a good that's a

good question so is structural learning possible in graphs like these um so that is actually something that is still an active field of research that would be awesome if you can automate also the structure learning of the graph itself but there are a lot of challenges there so how do you parameterize this graph structure um or and and how do you do how do you do that actually right what is your search space in terms of graphs how do you how do you learn node functions if you don't have a node function given for your graph right so these are all very difficult questions and you don't have a straightforward answer so so that's why this actual model design step in the design cycle itself is still kind of a creative process there is an engineer that has to come up with a model and as an engineer you also have to think about how to adjust that model if you're not satisfied with it so if free energy is still high after you run your agent well something is going on something is wrong with your model it's not an accurate representation of what is the environment and then you might adjust that how do you adjust that well currently it's still just trial and error so maybe you can say okay i think

here is still something that i can
improve maybe remove a node maybe the
model is too complex
[Music]
but structure running in itself is um
it is something that is
also it can be addressed so there are
there are methods like um like nested
model comparison and things like that
that you can do in fact graph so you can
for example if you have a nested model
compute savage dickey ratio and then
kind of prune your model based upon that
and so those are things that you can do
to find a complex model and see what you
can cut away
but adding to that model is still
difficult so once you've cut it away
well how do you add yeah i i don't i
don't really know so that's that's a
good question and uh i don't have a very
straightforward and switch you can
from my rattling so yeah
perhaps i can say something about it um
i mean i i totally agree with uh with
taste i mean generally
when you design a
system or signal processing system you
have to design the
structure uh you have to estimate the
parameters and then you have to infer
states right the states change very fast
uh it leads to a common filter and and
that's very
well known in factor graphs
then the next stage would be can we also
over the long term learn parameters
and our effectographs can do that pretty

well

now the the next stage in biology i mean
we know the structure is also alert over
even longer times right

and so that's not working at the moment
in formula you can compare like tests i
says if you have two examples you can
compare the free energy and just pick
the best one

but we are working on that and this is i
mean we're taking again the lead by uh
from carl's idea on bayesian model
reduction but how to implement that in a
factor graph is a research project so
we're we're we're

working on that and i'm not sure where
that will end but that's our goal did
you have phd position available bert so
uh

also to bring it back to the example of
the

motor car getting to an optimal
temperature it's like if every hour you
notice that there was a bump in the free
energy then you go back to the drawing
board and you you go to the people who
are on that car in that area and ask
well what happens every hour or maybe we
do need to include this other source of
information but the model that you had
does its job so you could include the
wallpaper and other features the room
but that's kind of this art and science
of engineering and that's why it's
always interesting to work because
often in research it's like let's finish
the analysis
but really there's this cycle that the

entire

model is embedded within

that helps us always keep an eye out for
those patterns that are

precluding being captured by our current
models

here's just

here's a second question from the chat

how does the factor graph approach work

if the goal temperature in this example

is not defined beforehand can the goal

be abstract so kind of a similar

question but rather than structure

learning how does preference and goal

orientation arise

again excellent question

so

that kind of

relates to the question well who sets

the goals

or how do you set goals

so

yeah

the plain answer to that is well it's an

engineer that defines that

um but it can also be a higher level

agent

that does inference and sets to his

goals

so then you get to kind of

turtles all the way down

argument in the sense well you can put

layer upon layer upon layer upon layer

and layers set goals and other layers as

the goals for that layer etc etc

um

so how that will work in practice um

yeah that's still again um

active research area

um

but in the end it should come down to
minimizing free energy

so so that's kind of the central theme
of this we want to we want to minimize
free energy

we do it by perception we do it by model
learning

um but also learning of the goals
should be driven by minimizing free
energy so you should choose the goals
that minimize for energy well how do you
do that

i don't know

you tell me right

yeah

one more question that was asked was how
or does this integrate with graph
databases

or with large data sets is this
something that scales or is already
integrated to work with that kind of
empirical data or

what

um i think it depends on what you mean
by large data sets

um

so

when i hear large data sets i usually
think about big data things like that

um

so um

this toolbox is specifically built for
dynamic modeling

right so

now in the sense that you have a lot of
features in your data but it's signal

processing so data might come in very quickly
and you have a model of how these data change over time

so

in that sense you want to do processing quickly so you want a lot you want velocity it's based on processing with velocity not processing with volume as for example other problems programming toolboxes are excellent at like um by torch and things like that right sampling base toolboxes

adword

all great toolboxes if you have a volume in terms of data and you have for example an iid model that you want to uh that you want to fit

um in our sense if we talk about large data we talk about volume so data that come in very quickly and we try to be good at that um phony lab is not yet that good at it to be honest um but reactive message passing is going to be the bomb

at those kind of of data sets

there's there's also a bit of that what we we really try to build these toolbox to build active influence agents and the idea that you just put these agents in an environment they make actions they select their own data so we don't really try to build a toolbox that is really good at just doing machine learning for a fixed set database

but rather we try to build
models that
let's say dynamic models that can adapt
and that can make actions and that can
in real time process streaming data and
and in that sense over time if you wait
long enough yeah there's a lot of data
streaming through it
but it was sort of generated by you or
at least affected by the agent itself
right
we don't try to optimize for
for big fixed databases rather for just
streaming data in an environment
it's a really fascinating point because
there's a lot of time dependent tasks
like autonomous driving where it's like
okay here's the 500 terabytes of video
or here's this now give me the best
possible score
in a time snapshot and it's it's
every single time with an active
inference agent we're always including
action and policy selection in the loop
and so
how how are we going to deal with that
but that's why the inspiration from
biological systems becomes so important
it's kind of like when people go oh well
the brain must have this processing
power and then if it were a light bulb
or if it was this kind of a silicone
computer it would take up this much
energy ergo's efficiency is such and
such but it's predicated upon
the kind of computers that we're used to
seeing instead of kind of starting with
what already exists and then looking for

an answer that
rests in that rather than how do we make
lower power transistors so that we can
fit them into the brain
so
awesome topics i think we're ready for
the next presentation
yeah hello everyone let me also share my
screen uh for a second
i hope it should work now looks good
so can you see my slides now yes
so okay so yeah hello again my name is
mitri bhagav and i want to introduce you
like this new reactive message passing
based framework for bayesian imprints in
july that we called reactive mp it's
more like our future vision on solutions
for active inference fornlab.jl is like
more like mature
framework it works and reactive mp is
our research project for now
and we are going to start with a
question
what is actually reactive message
busting
and in a
in most senses it's the same message
passing as a formula but re-implemented
in reactive paradigm so basically the
major issue with traditional approaches
in porn lab
is that enable in order to run inference
for our model we need to create this
shadow of messages in advance
so we need to pre-analyze our graph we
need to analyze everything basically and
we do we need to do it every time we
change our graph structure

and if graph is big this graph analysis
time and shadow creation may take a lot
of extra time
and it might be not a big issue but as
an engineer you may want to test a lot
of models until you're satisfied with
the performance and this extra delays
and shuttle creation times that they may
be just a bit of annoying and reactive
message passing last allows us to
eliminate this predefined shadow and
also it gives us a lot of other benefits
and possible future research directions
so instead of having instead of having a
fixed shadow we cast our graph to like
an event-based system
where everything can react on its
neighbor environment
and in terms of message passing
nodes react on incoming messages
and also it reacts on updated posterior
marginals over our parameters
and the whole model becomes reactive we
can also react on changes in this system
and basically do whatever we want so we
may perform some action
based on new updated posterior marginals
it also in message passing a natural
starting point uh is uh our data or
observations and the model itself reacts
on changes in our data or in our priors
and it also changes posteriors
accordingly
so here i outlined some extra benefits
from the active message passive
implementation so first of all we may
outline biological plausibility
because in nature we probably don't have

any predefined scheduler for our information flowing on neurons in our brain for example

uh they they are driven like by some chemistry like physics and in sense it's like also reactive system it constantly adapts the changes in an environment and also it reacts only if needed

and in terms of message passing based inference it may be unnecessary to even react in some events like some messages if they aren't really important

uh a fixed predefined shadow you're forced to do all computations even if you don't really need them

uh another great benefit of craft message busting is that it scales very nicely for very big graph so it's more like an answer for previous question

uh so it supports like hundreds of thousands of factor nodes even millions are possible uh i will show you an example in a couple of minutes but just to give you a bit of context so in this plot we met we modeled just some moving objects with known linear dynamics and the idea was to estimate the sedan states given noisy observations

in this example i used kalman smoother with 50 000 observations and this model contains roughly about 150 000 factor nodes so bayesian inference on this kind of a model is impossible on sampling based inference

but with reactive message passing it takes only about like eight seconds on just a home macbook

laptop

so we can go further a bit and make
reactive system robust and tolerant to
some let's say failing nodes or missing
data

from let's say failed sensor

we fix predefined shadow if something
like this happens we need to do
everything from scratch

we need to recreate our model we need to
again make a shadow

and it will just take some time

with reactive message busting we may
just stop reacting on some missing data
or maybe failing sensor

and we can just wait for them to be
available again

it makes it very robust in that sense it
also gives us an opportunity to change
our graph structure in runtime
and still perform inference without
stopping

so there's a slide about how we actually
do that

we use julia's main programming language
in our lab roughly about a year and a
half i believe ago

we wrote the library for reactive
programming in julia so it's completely
unrelated to message passing in just a
general framework for event based
systems

but that allowed us at the later point
to build reactive mp.jl package

and it implements free energy
minimization by message passing

we also introduced graphpl.gel package
that is a high level and user friendly

probabilistic model specification
language that we use in our demos
um yeah here i'm i
want to show you an example
just in a second
so you can you can see my screen right
yep maybe
one little size larger or two
i hope it will not break
yeah make it a little larger and then if
it looks unexpected you can pull back
okay here we go
better yep thank you
so here is our example so assume we have
some
moving object uh
it has some hidden states and just
assume that we know it's linear dynamics
for simplicity and we don't we don't
have a direct access to its location but
we have a noisy observation of this
moving object
and we wish to estimate the true
location of this moving object by only
observing its noisy measurements
and we may use a linear multivariate
gaussian state space model
like equations looks like this these are
equivalent notations they're the same
and basically here we say that uh okay
we have a state x at time step k
and it depends only on the previous time
step
through some linear operator a and we
also have a transition noise which is a
gaussian with covariance matrix b and
our observations are
uh

basically modeled also as a gaussian

with

covariance matrix q

so basically this is our model

and we can simply create a factor graph

out of it

and as you can see

that

our model specification resembles very

closely to the equations defined above

so here we have

our state x_k and it's modeled as like a

gaussian over previous state with some

let's say known covariance matrices

and this is how we build our model

and under the hood this

code it generates factor graph

uh and we can we can all later on use

reactive

mp.jail api to estimate hidden states of

our system but here's our example it

looks a bit off

probably because i zoomed in a bit

but it's fine i believe

and in this example i perform a kalman

smoother for 500 points and i just

decomposed the

trajectory of this moving object into

two axis x-axis and y-axis

and we can see that

common smoother predicted

like the real hidden states of our

system correctly

even though we have a lot of noise in

our system so these blue dots it's like

our noisy observations

and we go we can go further with this so

because our system is reactive we can we

can estimate uh our our states in real time

so here's our example let me try to run it

uh i hope it looks smoothly on my computer it's actually very smooth like but i i know that maybe recording is not that smooth like screen sharing but here in this example what we see is that okay we have an infinite data stream of a moving object the blue is a real one and the red is an estimated one and we can see that uh yeah let me reload it because it's kind of off

and we can see that reactive mp is able to perform and bayesian inference in real time and it actually adapts to changes in the environment and also changes the posterior over the current estimated state

and we can go even further you probably notice this orange line here just inactive for now but we can also incorporate a prediction for this model so we can just we can just extend a little bit our graph and we can also predict future state of our system like this

so now in this example i do bayesian inference in real time and i also predict the future state of my system like in orange and you can see that prediction also adapts to the new observations and yeah it changes basically its future

beliefs

over future states

uh yeah

let me stop it and continue with the
presentation

so um

yeah i'm going to talk about

like when reactive message passing is

available actually it's already working

uh we have a fully working stable

backhand and api for exact and

variational bayesian inference we also

support expectation propagation but that

api is not yet stable so it may change

we support for now only conjugate models

from exponential family

we support extra constraints for

variation optimization procedure

such that form or factorization

constraint

so rocket.jl library it naturally

supports infinite data streams for

example from an internet or from some

sensor

and framework itself is able to handle

missing data

but that

api is also not yet stable

here

some of our future plans for this

platform so we want to extend it to

support non-conjugate models as well

basically foreign lab already supports

it

we just need to carefully port all of

the available existing functionality

from foreign lab to reactive mp

and we also want to integrate with other

probabilistic programming libraries that
exists in july community
reactive message passing gives us an
opportunity to
try to integrate parallel inference
and react in different parts of the
graph simultaneously using like
multi-core capabilities of our cpus
uh

we want to extend
graph ppl to support modular model
specification for now it's unfortunately
not possible
also reactive message passing naturally
gives us an opportunity
to run inference where different data
streams have different update rates
um what

that what i was talking about in the
very beginning about robustness but it's
also interesting how far can we push it
and maybe in it's possible to
automatically adapt model's graph
structure in runtime based on free
energy

but it's like a research project we
don't really know if it's possible or
not for now

uh we we have some ideas about
interactive visualization for message
passing based algorithms
and free energy minimization
visualization

and eventually we we are planning to
release this new platform as a stable
version of formula possibly
in 2.0

and that basically is everything i

wanted to show you in my slides and
thank you for your attention i would
have be happy to answer your questions
awesome first question was is there a
paper on this
reactivemp.jl or how does that work
there is but it's in progress so
it's not publicly available but
we are working on that so there will be
eventually paper about all of this
approach i will describe everything how
it works and how we use it so yeah i
mean of course if anybody writes us an
email who wants an early copy of the
paper i'll be happy to share it if it
stays
confidential i'd be happy to share it
it's very
interesting
development maybe to the other authors
like
what
other um
epochs of forney lab were there
um
how did we get with to this reactive
programming paradigm and also it seems
like you did a lot of the foundational
work in the lab
on the reactive programming
was that just in the julia
implementation or in the more conceptual
grounding
um
yeah we did actually really a lot of
work to support reactive programming in
julia basically julia is is like a young
language and it has no

like good capabilities to run reactive based like systems so basically we build it from scratch

yeah but we we used a lot of uh like let's say ideas from other programming languages that we had experience with

i mean i know that the active influence community uh matlab and matlab is fantastic but for

for really for if you want real time processing of streaming data then julia is a very good

is a better option because julia is almost has the syntax of matlab from a user's viewpoint

but out of the box it's almost as fast as c

and so it was a better combination for if you are in engineering and you actually want to build systems that run in real time you want to make demos and julia is a better language also uh dimitri also makes use of some really advanced stuff in julia it's called multiple dispatch but it doesn't matter

what it is but it's it's quite advanced and it's not

uh available in in matlab but it's extremely useful for what we do for do with the toolbox it's it seems to be

if you look at a graph and message passing it see it seems to be like oh this is actually not so hard to

implement

but there's a reason why there are extremely few toolboxes for factor graphs okay microsoft has built one infer.net and it's it's great but it's not really a real-time toolbox um and so as far as we know we are there's not much competition at the moment for us

not that our toolbox is very advanced yet but

um i don't see a lot of people working on this i see a lot of people working on multicarrier sampling but multi-carrier sampling will not work for the sizes of systems and for the real-time data streams that we want to do with active inference right we are working with systems where we want to make actions and influence our data so this is a real time system

um

and so we have to go to want to actually scale up we have to go to

build a really high quality professional toolbox

that automates message passing i hope this will be uh one of those toolboxes and hopefully will also be other toolboxes but

yeah that's why we're doing this

very interesting answer

how has the

dialogue between

the math and programming and computer science side and then the active inference side been like what has

each side kind of contributed because it sounds like some of those real time insights of active inference are kind of propagating back to the design of algorithms and then

we see both directions so how has that played out maybe for any of each or each of you

i can also answer maybe this question from my point of view i mostly like doing programming uh

but i still learn something new every time because we like in this constant uh dialogue with

math guys in our lab let's say and i learn something new every day and of course it reflects like our design choices for our software as well so it's kind of but sometimes it's it's very hard to fit this high level mathematical ideas to the actual implementation

and also it's very hard to make it efficient

so

yeah we are not uh we're not done with this um i mean the toolbox is available for anybody and and

all the nice territory examples the basic thermostat the tic tac toe and it all works very smooth

um

our

hope is that with reactive message passing we can actually

you know get a

path to scaling this up to to serious applications

and that i mean that's real i mean we're
an engineering group we want to build
systems that really do something
useful i also work for a hearing aid
company so i want to build real time
audio processing algorithms
and
and others may want to use this for
robotics so
yeah but and that's not going to work in
in real time model up it's not going to
work
um yeah a bit with all the multi-card or
something so
this is our effort
and it's quite
it's takes a long time because it's very
multidisciplinary efforts right we have
in in my group we have a mathematicians
and
neuroscientists and
computer scientists because it's it's
it's it's difficult
there's a lot of different
expertises
that that you need to build a good
toolbox for active inference
i think because it's so difficult there
is a lot of like
um like cross
fertilization between the active
inference community and the engineering
that we do
the community is mostly interested in
explaining biological systems right the
the energy principle and active
inference as
a model for biology

and i think that's very interesting
because while nature has a certain way
of doing things
efficiently um
and if this is a model for that
then it's also a good idea to take that
to engineering
and that's kind of our recommend so we
kind of take the ideas that are
available in the active inference
community and where they explain brains
and think about brains
and think hey how can we take these
ideas
and
use them to build an engineering system
and i think that's kind of the main
yeah the main interaction between it and
hopefully then we build tools that
the community can also use then in their
research eventually
what that made me think about was
it's really a reframing of some of the
main challenges like seeing signal
processing as a real-time event or
seeing
the causal relationships between action
and future data all these kinds of
reframings of the problem it's not like
the active inference
algorithms
take massive matrix uh calculations to
do per se they can be very simple or not
but it's actually that reframing in the
the embedding of action within every
step
that then ends up solving the scaling
some of the scaling challenges some of

the resilience challenges because like
resilience of a
transportation network in a city they
would do
sampling or they would take something
that's an unfolding dynamic process and
then try to roll out a million
iterations of it statically
so it's again it's not that the
calculations
inside of the loop have to be
challenging it's just a reframing of
prioritizing action
ended up
going down these roads of scaling and
real-time capacity
yeah i i like that
the thought um perhaps i can say
something about it in the
context of what i work on because i work
on designing basically i'm a hearing aid
engineer i work on
[Music]
and if you ask a hearing aid engineer
signal processing engineer so what is
your task
well our task is to build the best
hearing aid algorithm
and
so
then what happens is that
a hearing aid client goes to the store
and buys a hearing aid usually
it's very happy and goes out and then
two weeks later sits in the restaurant i
cannot understand
her conversation partner because
there's noise and this wasn't expected

and there's nothing she can do
because you can't ask a client a hearing
aid client to
to

fiddle with hearing aid parameters so if
this happens
a few times

um then she throws or she puts her
hearing aid in the drawer and this uh
indeed about 10 20 percent of hearing
aids end up in the drawer they're very
expensive and

[Music]

20

ends up not being used a really
sad statistic
so rather

you can you can turn it around and say
okay what is the real hearing aid design
problem the real hearing aid design
problem is send somebody out with any
hearing aid but what do you do when
she's unhappy in the field
at in the restaurant

and what we want to build is an agent
that will

then so she just slaps her wrist raceway
i'm happy and now this agent needs to
make an action and give her new
parameter settings that are
the most interesting for her and and and
and the in the best

compromise between information seeking
and goal-driven gold riven being making
her happy

so

hearing a design from my viewpoint now
is just build an agent that will

make actions make hearing aid proposals
when she's unhappy
and if she
right she's unhappy and then hearing a
mixed proposal says no that's not good
well the agent gives another proposal
and that is okay that's better and then
we move on and maybe a week later the
same event and this goes on continually
but there is at least
a procedure to keep moving on to keep
improving over time right
so that's
the the real thing of design is the is
then the the action in the field in c2
which is very different from what's
currently happening which is hearing aid
engineers sitting at the desk
with model up not in the environment
so that's a it's it's a paradigm shift
and active inference agents
can maybe make that happen and could
really
i think uh
create a pivoting point for something
like signal processing design
which is uh
you know it's not something that you
think about when you're a neuroscientist
but this could really mean something big
for engineering
and not just for signal processing but
also in different engineering
disciplines and
so that's why i think over time you'll
see more and more people coming from
different fields
getting an interest in this uh

active inference and the free energy
principle
uh even even uh non
the not neuroscientists right but
engineers yeah
yeah cool we hope so and and that really
reminds me of the pragmatic turn
which is it's a little bit like a
horseshoe theory with the engineers and
the philosophers
often talking about pragmatism
in different ways different communities
different tools
and now there's a way to kind of close
that gap or at least map across it
where we can embed some of those
insights about
really an activism the role of design
science anticipatory design science with
take it or leave it philosophy or
there's the toolkit to develop
maybe it's interesting five years after
working with a toolkit or maybe five
years of philosophy and then you're
curious about the toolkit
so one
question was
how do these graphs account for
temporally deep models how are those
specified
or what
make what is different about the graph
because it's just calculating the next
action to take in the examples that you
provided
maybe guys you can show your graph again
how how the future is represented
um and how do you add temporal thickness

to that for example that's kind of so if
you have if you have a graph
like
chair screen yes
if you have a graph like this then how
do you add temporal thickness for
example
um that's that's just about adding
layers
so
what is shown here is one layer
and this layer
it acts
at a certain clock time you know
observations come in discretely
um
but you might have a layer above this
that is somehow connected to this layer
below
that acts at a higher temporal time
scale
so it evolves slower
but it regulates or influences the
parameters
of for example a transition model here
or an observation model here
so you get a time varying
observation model that is influenced
by
a slower evolving layer
on top of that you can get an even
slower evolving layer right and that's
that's the way
you can build hierarchies
um
and actually there's um also a pg
student in our group ismael senos who
has done some very interesting work on

that and he has investigated how you
make hierarchical
gaussian filters um so hierarchical
gaussian systems where you have one top
layer influencing the variance
of the transition model of the layer
below
and
you can see that you can get
can model very natural signals with that
so so in nature you have you have
signals that are time varying the
statistics there their time varying and
the statistics of those models
are time varying
so a hierarchical model in that sense
can be very useful for that and of
course that's that's yeah of course
also how our brains are structured so
again there you have this inspiration
from nature
um on how to structure
the models that we try to build
um
with vector graphs
at least in theory they should be pretty
easy
because you can just connect them you
can connect layer on top of that and and
see how it behaves
and eventually the complexity of your
model will become very high and you will
get penalized for that automatically so
there will be a cutoff point
um given your data and that will be the
optimal amount of layers that you need
in order to explain away all the
variants

that you observe
so so that's kind of how we think about
temporal thickness
and and
time dependence and dynamical models
like these
then can i share my screen also yep
sure okay yeah let's see
because this um awesome
yeah
this is a graph from a paper
a few years back uh in frontiers
um it's called i think uh
deep temporal models inspect factor
graphs or something
and here you see a three layer system i
mean don't bother about the details now
but here you see at the top layer just
just basically has one one section one
time step
and in that time step at the top layer
you have
two
observations or two steps at the
middleware
i need uh yeah
i'm not sure if you can see my uh
okay and then again for the let's say
the third layer whenever the middle
layer tips takes one step we have two
steps here so so there is sort of a
finer granularity
at each layer right and so we can build
hierarchical
uh models really and uh so we're doing
that also in uh i mean also yeah if you
you
you look at some of the papers on our

website you'll find a lot of babies on
the hierarchical gaussian filter from
chris matisse we've implemented that
in in
but also um
i mean you can also do it for active
influence things like uh and it will
look
like this uh
thomas pyre has also graphs like this in
some of his papers
very interesting and we see a lot of the
same variables like g
d b
like we can it's kind of a different
representation of active inference so
yeah
this is kind of also going back i think
to the earlier frontiers paper what are
some of the big equivalencies
or kind of like airtight mappings
from into
forney factor graphs
because there's probably some broad
areas of application
that could almost be hot swapped for
this underlying it perhaps
so what like are the equivalencies with
just bayesian graphs in general versus
factor graphs versus other
topics at that level
i think a foreign style vector graph or
a bipartite graph or a beijing network
they aren't really different
they can be used as
they're just a representation
of a factorized model
and the way you represent that

can have an impact about how you think about those models

so for example if you choose a 4-install effective graph representation it's very suited for signal processing because you kind of can see these messages as signals that flow over your graph

so for us as engineers

this is a very intuitive representation

in bayesian networks it's a bit more the model representation itself is more compact in that sense that you only have variable nodes and you see how they relate so it gives you a good idea of model structure

so it can be very nice to

have a quick overview of model like that

and then you have for example a bipartite graph which also shows a relation between variables as an additional kind of

vector node in between

um and it goes it gives you a bit more granularity on how these are connected and gives you room to talk about well what is the relationship between these variables

and so in the end they're equivalent you can take one model and represent it in three different ways maybe even more but i think it does impact the way how you think

or how you think about these models so um

that

that usually comes up when writing a paper

and then you have to think okay what is

actually the best representation for my
idea

sometimes it's about budget graph

sometimes it's a visual network

sometimes it's for install photograph

um

it just depends by your story yeah

hopefully

i do find though i mean in my experience

when we think about systems and a whole

set of equations if you read that in a

paper

or if you write it down um it gives more

insight if you draw to me it gives more

insight if i draw the graph

and

there is an x

there must be an exact correspondence

between the

the graph and the equations and very

often if you draw the graph and then you

write down the equations there's not an

x-axis corresponding so you learn from

it you you you basically you know there

must be an exact correspondence and

basically you you often find out you

have an error in your equations or in

your graph

but i think uh

like the the end goal the end goal uh

is of to make the toolbox a toolbox for

let's say for the community to design

their own active influence agents and

what i

envisioned there is something like

simulink i'm not sure or labview i'm not

sure if people are familiar with that

but um

these are graphical models you want to
actually also define them graphically
and you want to have a palette with
notes and just
draw your graph and say this is my model
and now go run
you don't want to worry about the
inference the inference is under the
root dependence passing should be done
by
the designers of the toolbox but you
should just be thinking about your
graphical structure that's the uh
and this one and connect it
with your mouse to your microphone of
your computer or to your camera
and maybe to uh
you know there's also connectors in the
toolbox for robots and then
it should just go right
that's going to be the bob ross of
graphs
that's how you want to design i think
right you just want to draw the brain
and at least the generative model of the
brain and just let it go
and don't worry about how inference
takes place
what what you just said there about
viewing the equation and then maybe
going from the pen and paper to the
programming language to the graphical
representation and then kind of
cross-checking it's like when you
reverse translate languages and you're
when you lock in where two words map
back to each other you know you've
you've made a map but if you're on this

infinite loop you're you're lost in the
word space so when you move across
sectors like that or across modalities
with the
analytical the simulations on the
computer and then the graphical it kind
of embeds action
in what otherwise might be seen as a
knowledge
product a product of inference and then
a final action
and that's a similar fallacy to
train the dynamic process in a snapshot
and then expect that it's going to work
in real time so it's like embedding this
real time flow
in that the production of knowledge yeah
yes yes we are important yeah yeah
um yeah we're implementing active
insurance while we are trying to design
this stuff that's true yeah
yeah one one general question was just
about factorization
so
the factorization the
way we go about it is starting with our
intuition and looking for residuals or
how of all the ways to factorize a model
do we find one that works
yeah there's two kinds of factorizations
there's a factorization in let's say in
the generative model
right that's the graph that we draw
and so i mean let's say most people
uh or the most common models are these
these
these markov models right where you you
kind of retain a current state that

summarizes everything that happens in the past

uh and you use that current state to basically to yeah to summarize the past and then that's all the information have about the past so you don't need to remember the past you just remember your state

and with that and you did make a new observation you combine the information make a new state

and so forth there so hidden markov models

palmdps all these models have that same structure these these this microstructure so that's for the generative model of these dynamic systems

then there is a second question if we now do inference

um and um yeah then there's often what we call the mean field assumption for the variational posterior but there are also

variants on that structured green field so you can you can

still decide if you want your posterior to be even more structured or let's say even more factorized than the generative model

but in the end yeah it's a

it's a proposal

uh

you just run it and if you have another proposal you just run it and the one that has the lowest free energy wins that's uh that's it right the challenge is to automate that process

go from the
let's say the poorer structure to the
better structures
just by failing to minimization just by
message passing in real time without
stopping the whole process doing an
analysis
right it should just keep moving right
it should just just structure adaptation
should be like state estimation it just
keeps moving over time there's no
resetting
maybe there's a bit of a
thing like
a dreaming stage right but
the principle time moves on
and that's the challenge and we don't
know that in our factory class but yeah
we
i mean
we have one hd student who is currently
looking into that
yeah like as an engineer i think um the
question is well what what do you start
out with what will be your initial model
proposal uh so if you talk about
factorizations of your generative model
you think uh every factor then it kind
of represents a prior distribution or
conditional distribution and that's how
you kind of build up your model
where do you start
and it starts with how you believe
the world generates observations
in that sense you think about
what is the causal structure
of the environment and you might have
some idea about how physics works

um or how
states transition in in your environment
or at least how you believe this
transition and that's kind of where you
start
and you think okay well
maybe let's try a very coarse note this
minus one for example in my talk let's
try that here because i don't really
know what i want to put there but i have
to put something let's try something
and see how that works it kind of models
how i believe the world works
and that's that's how you start out
and then you start thinking well i
actually know a bit more about it i'm
not satisfied with how this model
proposed how this model works um and i
can change this minus one for example
with uh with something that's a bit more
complex because i know the physics i
know how
these uh
how temperature integrates with distance
for example and i can build that in
um so that's how you go to a second
proposal
and it's always inspired by
uh how you believe
the causal structure of the world is um
is uh yeah is
how that works so in that sense it's
kind of theory building you're building
a theory or an explanation of your
environment
and
well maybe if you're really really good
if you have an excellent model then you

can find something that
kind of improves upon the state of the
art because you can always improve
just find something that gives you a
lower free energy or better model
performance in that sense
um and you can keep tweaking and
tweaking
um
these models and these factorizations
and
things like that
yeah
free energy
minimization
has been described or just as a way to
rank the different models and as an
imperative on these multiple different
fronts how do we know that
free energy minimization is making the
policy
that
is going to be like resilient is there a
way where similar to a local
optimization getting trapped somewhere
in a bigger optimization space
there might be some avenue of free
energy minimization that
makes the system just like break down
like we'll get there faster let's just
accelerate and then
there's some
kind of one-time failure of the system
i'm just wondering how can one
metric that's a number you know and can
be sorted how can it evaluate such
radically different
chess strategies or driving

approaches at the end it's just
probability theory and probability is
also just a number right
and the energy kind of
it approximates your evidence which is
the probability according to your model
of observing the data so if you make a
model that gives you the best evidence
then it's a good explanation for for
your for your data
so in the end it's just
you try to do
approximate probability theory
um
and free energy
is then bound upon your evidence and it
also takes into account
this posterior divergence term which
kind of
says the price that you pay for
approximating bayesian inference
so it has two parts to it it has
um the model evidence part
and it has the posterior divergence part
and both play a role one says okay um
this is
the quality of my solution it's it's how
well
uh
your model explains the data that you
observe you can get you can get a number
for that the surprise or the negative
log modal evidence
and you have the other part of stereo
divergence that says
well this is the price and information
that you pay for making this
approximation you can also put a number

on that

it's just a chaotic versions that you

evaluate

um and then you add those two and you

get a number

um

so yeah so the question is how can one

number kind of

represent

everything in terms of quality

well you might care about different

things than how well your model predicts

or evaluates the observations right

and then if you care about other things

um

then then

yep then then you might use different

number but

um in our in our sense in our in our uh

in our work

yeah we're kind of

uh

we want to have this bayesian measure of

quality

yeah

i don't know if that really answers your

question but it's it's it's still a good

question it's um

it's kind of is is free energy or that

number that you get is that enough

right

that's that's kind of what's behind this

and probability theory says yes but if

you apply it in practice then you care

about maybe other things you care about

how many people survive before how many

you know

how many how many mistakes you uh you

you cash from from something and if you
care about that well then you should use
that as a performance metric
i i think right i mean it's uh
the energy of course has has has many
decompositions right in in in
complexity minus accuracy and
and and surprise minus uh
minus the kl divergence
um but
to
i'm not sure if i interpret the question
correctly but if you say like how do you
know that it does well
in in practice in the field
well you don't right um
the
and look the only uh
thing you can say is that i mean what
the system does it will look for
a configuration that minimizes free
energy but you don't know what you don't
know you don't know if there is another
model that would do better if we haven't
simulated that model then we don't know
and um
the only learning opportunities there
are
is to generate errors so um
the only way to really improve the model
is to actually
hit indeed um let's say situations where
it's not working
and then you need to adapt your
parameters and and
and even more over time you have to
structure so
yeah it's uh

i mean building a good system is a process
right it's not a matter of uh just
you have to be extremely lucky if you just
build a system that and it completely works right
uh
it's what this
this between this orbitals active influence describes is a process towards a better
systems but not
it doesn't describe a system that doesn't make mistakes in fact it needs the mistakes to learn
and it describes the process
awesome points it's that operational insight that if you're getting 100 on the test like something is
at the very best less than informative
at the very worst you're you're way down the wrong path and um and then it reminded me of like a grocery store is
it that we're ranking the different objects according to one measure
no not really it's like balancing strategies
for
finding preference and that's where all of these assumptions like the factorization and the mean field and
it's not guaranteed to give you the best object every time and there's still the stochasticity of the real world and the ability of the grocery store to change
like it's not the end of the story

it's actually just

a practice in a process that

has some of those features of reactive

graphs and biological systems

that's right yeah

well um if any of you have any final

comments this was just really one of the

coolest times to learn about this

but

you're always welcome back to join any

time as a participant or to present and

if you have any final comments you can

note them

well it was a

real pleasure to be here and uh

if we have new stuff we would love to

come back i think it's a fantastic

fantastic show here or show it's a

forum or whatever it's properly called

but i really enjoy it and we really

enjoyed being part of it

and uh thanks a lot for the discussion

some some food for thought there as well

it's really cool

good question for every opportunity to

present yes

great

okay until next time bye